

**TEMPLATE
PROJECT WORK**

Corso di Studio	INFORMATICA PER LE AZIENDE DIGITALI (L-31)
Dimensione dell'elaborato	Minimo 6.000 – Massimo 10.000 parole (pari a circa Minimo 12 – Massimo 20 pagine)
Formato del file da caricare in piattaforma	PDF
Nome e Cognome	Alex Di Paolo
Numero di matricola	0312300365
Tema n. (Indicare il numero del tema scelto):	5
Titolo del tema (Indicare il titolo del tema scelto):	Machine Learning per Processi Aziendali
Traccia del PW n. (Indicare il numero della traccia scelta):	18
Titolo della traccia (Indicare il titolo della traccia scelta):	Triage automatico dei ticket con Machine Learning
Titolo dell'elaborato (Attribuire un titolo al proprio elaborato progettuale):	AutoTriage NLP: Sistema intelligente di classificazione e prioritarizzazione ticket per l'assistenza aziendale

PARTE PRIMA – DESCRIZIONE DEL PROCESSO

Utilizzo delle conoscenze e abilità derivate dal percorso di studio

(Descrivere quali conoscenze e abilità apprese durante il percorso di studio sono state utilizzate per la redazione dell'elaborato, facendo eventualmente riferimento agli insegnamenti che hanno contribuito a maturarle):

L'idea alla base di questo elaborato nasce da un'esigenza molto concreta delle aziende: ottimizzare i tempi di risposta e la gestione delle richieste nel supporto clienti. Partendo da questo presupposto pratico, ho sviluppato il progetto **AutoTriage NLP**, un sistema basato sul Machine Learning pensato per automatizzare lo smistamento dei ticket. Nel realizzarlo, ho cercato di bilanciare le sfide tecniche dell'elaborazione del linguaggio naturale con le normali necessità e i vincoli rigidi di un'impresa reale. Per arrivare a un risultato funzionante, ho curato l'intera architettura software, dalla preparazione dei dati fino alla creazione di un'interfaccia utente interattiva.

L'obiettivo principale del progetto è dimostrare a livello pratico come gestire, manipolare e analizzare dati testuali non strutturati utilizzando Python. Fin dal principio non volevo limitarmi a un semplice esercizio tecnico, ma ho cercato di far convergere in questo lavoro gran parte delle competenze multidisciplinari apprese durante l'intero corso di laurea. Scrivere il codice ha richiesto infatti di fondere insieme diversi insegnamenti.

L'esame di **Programmazione 2** è stato l'insegnamento centrale, dato che il progetto è scritto interamente in Python. Ho applicato quanto studiato sulle strutture dati e utilizzato ampiamente librerie come Pandas e NumPy per pulire il dataset scaricato da Kaggle, isolando la logica di pulizia nel file **prepare_data.py**. Sempre grazie a questo corso, ho usato Matplotlib e Seaborn per renderizzare i grafici della dashboard e ho implementato blocchi **try-except** per gestire in sicurezza le eccezioni nella pipeline di traduzione. Anche **Programmazione 1**, seppur fosse un esame incentrato sul C, mi ha lasciato l'impostazione logica per affrontare il problem solving. Ho riutilizzato quell'approccio strutturato per progettare la pipeline di preprocessing, organizzando il codice in moduli separati all'interno della cartella **src/** per mantenerlo pulito e leggibile. Lo studio di **Algoritmi e Strutture Dati** mi è servito per scegliere le strutture più adatte, come dizionari e hash map, per gestire i vocabolari in modo efficiente. Inoltre, per evitare che la traduzione in batch di CSV molto grandi diventasse un collo di bottiglia, ho parallelizzato le operazioni sfruttando il **ThreadPoolExecutor**, mettendo in pratica per la prima volta su un caso reale i concetti di complessità computazionale.

Oltre alla stesura delle funzioni, ho cercato di scrivere il codice pensando fin da subito alla sua manutenzione futura e scalabilità del sistema, fondendo **Ingegneria del Software** e **Basi di Dati**. Il primo mi ha dato il metodo di lavoro. Ho diviso il progetto in layer distinti separando le competenze di Data, Model e Frontend, e ho utilizzato i concetti dei design pattern per creare la classe **UnifiedModel**, capace di gestire i due classificatori paralleli mantenendo l'interfaccia pulita per l'utente. Anche se ho usato file CSV invece di un database SQL tradizionale, ho comunque dovuto applicare le regole di normalizzazione per ristrutturare il dataset originale in quello che poi è diventato il file **tickets_it_augmented.csv**. Questo mi ha garantito di far addestrare l'algoritmo su dati di training coerenti, integri e senza duplicati.

Naturalmente, niente di tutto questo avrebbe funzionato senza una solida componente di matematica e statistica, poiché i modelli predittivi utilizzati si basano su fondamenti teorici ben precisi. Studiando **Calcolo delle Probabilità e Statistica** ho potuto comprendere le metriche come la TF-IDF, o capire come la Regressione Logistica stima matematicamente la probabilità di assegnare un ticket a una specifica categoria. Allo stesso modo, le metriche che ho generato nel report (Accuracy, Precision, Recall, F1-Score) e la lettura della matrice di confusione derivano direttamente dagli argomenti di questo esame. **Matematica Discreta** e **Analisi** mi hanno permesso di capire cosa succede quando il testo viene vettorizzato o quando l'algoritmo calcola le distanze tra le varie categorie. Perfino l'integrazione della libreria LIME, usata per rendere il modello interpretabile, si basa sullo studio degli spazi vettoriali.

L'ambito di **Tecnologie Web** è stato il tassello finale per dare vita a un applicativo vero. Ho sfruttato le conoscenze base di HTML e CSS per personalizzare l'interfaccia grafica tramite il file **style.css**. Inoltre, aver chiaro come funzionano il protocollo HTTP e l'architettura Client-Server mi ha permesso di capire come il framework Streamlit gestisce le interazioni dell'utente e come integrare le API esterne di traduzione senza interrompere il flusso dell'applicazione.

Infine, ci tengo a evidenziare che il mio progetto non si limita a essere un prototipo tecnico, ma mira a risolvere un problema di business reale rispettando le normative vigenti. L'insegnamento di **Diritto per le Aziende Digitali** mi ha reso consapevole dei vincoli del GDPR sulla privacy, per questo motivo ho deciso di non fare scraping di dati reali per evitare rischi legati al trattamento di informazioni personali. Applicando il principio di "Privacy by Design", ho optato per un dataset pubblico open source già anonimizzato, creando un sistema sicuro e "compliant" dalla base. Dal punto di vista di **Strategia, Organizzazione e Marketing**, sapevo che l'idea di un triage automatico avesse senso se fosse in grado di portare un reale vantaggio in termini di efficienza. Per questo ho inserito un calcolo del ROI operativo all'interno di questo elaborato proprio per dimostrare come questa automazione possa ridurre i colli di bottiglia e abbattere i costi operativi, unendo l'aspetto tecnico a quello aziendale ed economico.

In conclusione, l'obiettivo di questo elaborato è stato applicare le nozioni teoriche a un caso d'uso pratico. Collegare lo studio della teoria statistica alla scrittura del codice mi ha permesso di ottenere un prototipo funzionante, verificabile e ancorato a dinamiche aziendali realistiche.

Fasi di lavoro e relativi tempi di implementazione per la predisposizione dell'elaborato
(Descrivere le attività svolte in corrispondenza di ciascuna fase di redazione dell'elaborato. Indicare il tempo dedicato alla realizzazione di ciascuna fase, le difficoltà incontrate e come sono state superate):

Lo sviluppo pratico del progetto mi ha impegnato per circa quattro settimane, per un totale stimato di 80 ore di lavoro. Per gestire al meglio le tempistiche ho preferito strutturare fin dall'inizio l'attività in cinque macro-fasi.

Fase 1 - Scelta della traccia, ricerca e setup (circa 10 ore)

Inizialmente ho analizzato le varie tracce proposte e ho scelto di dedicarmi al progetto "AutoTriage NLP" perché mi sembrava il caso d'uso più facilmente spendibile in un contesto aziendale reale. Prima di scrivere codice, ho iniziato raccogliendo il materiale sulla biblioteca online di UniPegaso, in particolare su Pearson Bookshelf, ripassandomi per bene le dispense di Programmazione 2. All'inizio temevo di allargare troppo il progetto e di non rientrare nei tempi. Fortunatamente, le sessioni di didattica interattiva con i docenti mi sono state di grande aiuto per definire dei confini chiari e focalizzarmi sull'ottenere prima di tutto un prototipo che funzionasse a dovere. Nel frattempo, ho configurato l'ambiente di sviluppo su Antygravity.

Fase 2 - Analisi del problema e preparazione dei dati (circa 20 ore)

Volevo evitare di usare dati generati casualmente o con dei template fissi, perché avrebbero reso il sistema poco verosimile. Così ho recuperato da Kaggle il dataset “Customer Support Ticket”. Essendo interamente in lingua inglese, ho dovuto ingegnerizzare lo script **prepare_data.py** per tradurre massivamente oltre 20.000 ticket in italiano sfruttando le API di **deep_translator**. Qui ho incontrato la prima vera difficoltà tecnica: c’era il rischio concreto che la traduzione automatica stravolgesse il senso dei termini tecnici (ad esempio “billing issue” tradotto male). Inoltre, interrogando le API in loop, per evitare blocchi dovuti alle troppe richieste ho implementato un sistema di salvataggio intermedio (caching) e applicato dei filtri basati su espressioni regolari (Regex) per ripulire il testo. Infine, ho mappato le vecchie etichette di Kaggle sulle tre categorie di mio interesse (Amministrazione, Tecnico, Commerciale), ottenendo il dataset finale **tickets_it_augmented.csv**.

Fase 3 - Sviluppo del modello di Machine Learning (circa 25 ore)

Questa è stata la fase più lunga e complessa, concentrata nel file **train_unified_model.py**. Dopo aver sistemato la fase di preprocessing del testo, togliendo punteggiatura e le stopwords, ho strutturato la rete logica del modello. Piuttosto che addestrare due classificatori separati, ho creato una classe **UnifiedModel** basata direttamente su un **MultiOutputClassifier**. Questa decisione mi ha permesso di far calcolare al sistema la Categoria e la Priorità in modo simultaneo, ottimizzando e alleggerendo i tempi di calcolo. Ovviamente, lavorando con dati reali, ho dovuto gestire un forte sbilanciamento delle classi, che rischiava di portare subito il modello in overfitting. Per affrontare il problema, ho diviso i dati con un classico split 80/20 per il training e il test, monitorando costantemente metriche come Accuracy e F1-Score e analizzando le matrici di confusione per capire dove l’algoritmo facesse più fatica a distinguere le categorie.

Fase 4 - Interfaccia Web e LIME (circa 15 ore)

Ho sviluppato una web app reattiva usando **Streamlit** all’interno di **app.py**. Il mio scopo principale in questa fase era integrare la libreria LIME per rendere l’algoritmo “interpretabile”. Spiegare a un utente non tecnico perché una macchina ha preso una certa decisione è sempre complicato (il classico problema della “black-box”), ma LIME mi ha permesso di generare grafici che mostrano visivamente all’operatore quali specifiche parole (es. “fattura” o “virus”) abbiano spinto l’algoritmo verso una determinata scelta. Oltre a questo, per rendere il sistema ancora più solido a livello pratico, ho inserito una logica ibrida per il calcolo della priorità che unisce le previsioni statistiche a regole di sicurezza fisse, basate su parole chiave in grado di forzare l’allarme se il modello statistico dovesse fallire.

Fase 5 - Test, validazione e stesura del report (circa 10 ore)

Infine, l’ultima fase è stata dedicata al collaudo del software. Ho effettuato dei test funzionali sull’interfaccia grafica inserendo di proposito ticket brevissimi, ambigui o senza senso logico, per assicurarmi che il sistema gestisse correttamente gli errori. Solo alla fine sono passato alla stesura dell’elaborato, che ha richiesto un’attenta formattazione dei dati estratti dai vari test (le matrici di confusione e le metriche) e l’organizzazione discorsiva di tutta la documentazione finale.

Risorse e strumenti impiegati

(Descrivere quali risorse - bibliografia, banche dati, ecc. - e strumenti - software, modelli teorici, ecc. - sono stati individuati ed utilizzati per la redazione dell'elaborato. Descrivere, inoltre, i motivi che hanno orientato la scelta delle risorse e degli strumenti, la modalità di individuazione e reperimento delle risorse e degli strumenti, le eventuali difficoltà affrontate nell'individuazione e nell'utilizzo di risorse e strumenti ed il modo in cui sono state superate):

Per la realizzazione di questo elaborato, ho dovuto compiere delle scelte architetture ben precise, selezionando strumenti e librerie che mi garantissero il miglior compromesso tra le richieste didattiche e l'effettiva utilità in un vero contesto lavorativo.

Il primo problema è stato trovare dei dati validi. Per addestrare il modello avevo bisogno di dati verosimili. Invece di generare ticket fittizi che avrebbero solo introdotto distorsioni e bias nel sistema, ho preferito usare un dataset pubblico di **Kaggle**, denominato “**Customer Support Ticket**”, contenente oltre 20.000 richieste di assistenza reali e già anonimizzate. Questa scelta mi ha permesso di lavorare su un caso di studio statisticamente valido e di evitare in partenza le problematiche legate alla privacy e alla raccolta di dati sensibili aziendali. Tuttavia, il dataset originale era tutto in lingua inglese. Per risolvere il problema, ho utilizzato le API di Google Translate richiamandole tramite la libreria **deep-translator**. Non l'ho fatto solo come semplice traduttore, ma l'ho sfruttato come un vero e proprio strumento di **Data Augmentation**, che mi è servito per convertire e adattare l'intero corpus testuale in italiano, rendendolo coerente per il training del mio modello.

Per consolidare le basi teoriche e pratiche mi sono affidato a diversi testi accademici e manuali, consultati principalmente tramite la piattaforma universitaria Pearson Bookshelf. In particolare:

- Gaddis, T. (2016). *Introduzione a Python*. Pearson.
- Zinoviev, D. (2017). *Data science con Python*. Apogeo.
- Maggi, G. (2020). *Data science con Python*. Edizioni LSWR.
- Marin, I., et al. (2019). *L'analisi dei Big Data con Python*. Tecniche nuove.

Ovviamente ho consultato anche la documentazione ufficiale online delle varie librerie per capire come implementare le funzioni.

Passando allo stack tecnologico, ho scelto **Python 3.10** come linguaggio base per la sua comodità nella gestione delle stringhe e la rapidità di sviluppo. Per tutta la fase di manipolazione dei dati mi sono appoggiato quasi esclusivamente a **Pandas**, è stato fondamentale per importare il file CSV da Kaggle, strutturarli in Dataframe ordinati, pulire il testo e gestire i complessi flussi di I/O tra la fase di traduzione e quella di addestramento. Per quanto riguarda la componente di intelligenza artificiale, piuttosto che scomodare framework molto pesanti, come TensorFlow o PyTorch, ho optato per **Scikit-Learn**. Seguendo sempre il principio della soluzione più semplice ed efficace, questa libreria si è rivelata perfetta e performante per un dataset testuale di medie dimensioni. L'ho sfruttata per gestire l'intera pipeline: dalla tokenizzazione delle parole alla vettorizzazione TF-IDF, fino alla doppia classificazione simultanea di Categoria e Priorità.

Per evitare l'effetto "black-box" tipico dei modelli di Machine Learning, ho integrato la libreria **LIME** (Local Interpretable Model-agnostic Explanations). Questo strumento mi ha permesso di rendere totalmente trasparente il ragionamento dell'algoritmo, mostrando visivamente all'utente quali parole esatte (es. "bloccato", "urgente") avessero pesato maggiormente sulla classificazione finale di un determinato ticket.

Per rendere il progetto fruibile a chiunque, ho creato un'interfaccia web utilizzando **Streamlit**. È stata una scelta strategica perché mi ha permesso di sviluppare una dashboard interattiva direttamente in Python, senza dover dispendere tempo e codice scrivendo e collegando file HTML o Javascript separati. In questo modo ho potuto concentrarmi esclusivamente sulla logica matematica del modello. Ho scritto l'intero codice poggiandomi sulla piattaforma Antygravity, sfruttando le sue funzionalità di autocompletamento e debugging rapido per snellire il flusso di lavoro. Per il versionamento ho utilizzato Git perché mi ha garantito una rete di salvataggio sicura, permettendomi di testare nuove funzionalità su branch separati senza rompere la versione stabile del software. E poi, l'uso di Git ha soddisfatto la richiesta di rendere open source l'elaborato. Tutto il codice sorgente e la documentazione sono consultabili sul mio repository GitHub all'indirizzo <https://github.com/dipaoloalex-dev/AutoTriageNLP>.

Durante lo sviluppo pratico ho dovuto affrontare un paio di criticità tecniche piuttosto rilevanti. La prima ha riguardato la fase di traduzione massiva del dataset, interrogando continuamente le API di Google, incappavo spesso in blocchi del server dovuti al rate-limiting. Per risolvere, ho dovuto ingegnerizzare da zero uno script di batch processing, inserendo dei ritardi programmati strategici (la funzione **sleep**) e una gestione rigorosa delle eccezioni (i blocchi **try-except**) per far sì che il processo di traduzione non si interrompesse a metà.

La seconda criticità è emersa in fase di visualizzazione sulla web app. L'integrazione tra Streamlit e i grafici generati staticamente con Matplotlib creava dei pesanti conflitti di rendering, specialmente quando i dati si aggiornavano in tempo reale. Studiando la documentazione di Streamlit, sono riuscito a risolvere il collo di bottiglia implementando un solido sistema di caching dei dati in memoria, utilizzando il decoratore **@st.cache_resource**. Questa singola riga di codice ha stabilizzato la visualizzazione e ha migliorato nettamente le prestazioni e la fluidità dell'intera interfaccia.

PARTE SECONDA – PREDISPOSIZIONE DELL'ELABORATO

Obiettivi del progetto

(Descrivere gli obiettivi raggiunti dall'elaborato, indicando in che modo esso risponde a quanto richiesto dalla traccia):

Quando ho iniziato a lavorare ad AutoTriage NLP, lo scopo principale era ovviamente rispettare le richieste della traccia, dovevo creare un sistema funzionante per automatizzare la classificazione dei ticket di supporto. Tuttavia, ho cercato di realizzare un progetto che fosse il più possibile vicino a un caso d'uso reale, prestando attenzione non solo alla precisione matematica dell'algoritmo, ma anche all'efficienza computazionale e all'interpretabilità del modello finito.

Per l'addestramento del modello, ho preferito evitare la generazione di ticket casuali o sintetici, che avrebbero reso il sistema debole, introducendo bias e risultando decisamente poco verosimile. Ho quindi recuperato da Kaggle un dataset reale, già preventivamente anonimizzato in origine. Questa accortezza mi ha permesso di garantirmi la totale assenza di dati sensibili nei CSV locali e di rispettare di base le normative sulla privacy. Poiché il dataset era interamente in lingua inglese, ho sviluppato uno script dedicato per tradurlo massivamente in italiano. Questo mi ha permesso di addestrare il modello su testi autentici, "sporchi" e complessi, molto più vicini al reale linguaggio utilizzato dagli utenti rispetto a delle frasi perfette.

Sistemati i dati di training, sono passato all'architettura vera e propria del modello di Machine Learning, dove anziché addestrare due classificatori separati e farli girare in modo parallelo per stimare prima la Categoria e poi la Priorità, ho deciso di ingegnerizzare tutto in un'unica soluzione. Ho implementato un singolo modello basato su una Regressione Logistica, andandolo a "incapsulare" in un MultiOutputClassifier. Questa scelta architetturale mi ha permesso di ottenere entrambe le predizioni simultaneamente con un unico passaggio di calcolo, alleggerendo il carico computazionale della macchina. A quel punto, le performance sono state valutate in modo rigoroso utilizzando metriche standard come Accuracy e F1-Macro. Ovviamente, queste metriche di validazione sono state calcolate a valle di una fase di preprocessing del testo, all'interno della quale mi sono occupato della tokenizzazione, della rimozione delle stop-words e di una profonda analisi degli n-grammi per catturare il vero contesto semantico delle frasi.

Un altro aspetto tecnico su cui ho dedicato particolare attenzione è stata la trasparenza e l'interpretabilità dei risultati finali. Nel Machine Learning non volevo che il sistema operasse come una "scatola nera". Per questo motivo ho studiato la documentazione e integrato a livello di codice la libreria LIME. Questo strumento analizza la singola predizione a runtime e mostra direttamente a schermo quali parole esatte (ad esempio i termini "server", "blocco" o "scadenza") abbiano spinto il modello statistico a classificare un ticket in un certo modo piuttosto che in un altro. A livello produttivo, ho reputato questa un'implementazione essenziale per permettere a un ipotetico utente di verificare il ragionamento del software e, soprattutto, di fidarsi del suggerimento fornito dalla macchina senza accettarlo ciecamente.

Infine, dopo aver sistemato tutto il backend in Python, ho dovuto curare l'aspetto applicativo e di usabilità del progetto per renderlo realmente presentabile e interattivo. Utilizzando le potenzialità del framework Streamlit, ho costruito una vera e propria web app che astrae totalmente la complessità del codice sottostante, nascondendolo dietro una UI pulita. Questo rende il mio sistema accessibile e utilizzabile anche da un utente assolutamente non tecnico. Ho fatto in modo che l'interfaccia permettesse all'utente due approcci di utilizzo: sia testare l'algoritmo in tempo reale (inserendo a mano il testo di un singolo ticket in una text area), sia di sfruttare l'apposito modulo di upload per caricare un intero file CSV ed elaborare uno storico di richieste in modalità batch. Quest'ultima funzionalità, in particolare, è quella che simula al 100% uno strumento software aziendale capace di abbattere e ridurre concretamente i tempi infiniti di smistamento manuale in un normale Help Desk.

Contestualizzazione

(Descrivere il contesto teorico e quello applicativo dell'elaborato realizzato):

Per inquadrare correttamente il mio lavoro, ho voluto inserire questo progetto nel più ampio contesto della Digital Transformation, concentrandomi in modo specifico sull'ottimizzazione dei processi aziendali legati al supporto clienti. Studiando il flusso di lavoro di un tipico servizio di Help Desk, mi sono reso conto di come emergano spesso criticità sistemiche, specialmente nella fase iniziale di triage, ovvero lo smistamento. È proprio qui che si creano i classici colli di bottiglia, si verificano continui errori di assegnazione manuale e i sistemi tradizionali faticano a rilevare le urgenze in modo tempestivo. Ho ragionato su come, in uno scenario convenzionale, l'errata assegnazione di un singolo ticket da parte di un utente comporti un faticoso re-instradamento manuale tra i vari reparti, che siano Amministrazione, Tecnico o Commerciale. Questo finisce per raddoppiare i tempi di latenza e aumentare inutilmente i costi operativi.

Per ovviare a queste inefficienze, ho pensato il mio prototipo come un middleware intelligente, capace di interporre in modo invisibile tra i canali di contatto (come form web o email) e il sistema di ticketing interno dell'azienda. Il modello analizza la richiesta in entrata e la arricchisce con metadati semantici, suggerendo automaticamente il reparto di destinazione corretto e il livello di priorità prima ancora che ci sia l'intervento umano. Questo approccio pratico mira a ottimizzare tutto il ciclo di vita del ticket, riducendo il Mean Time To Repair (MTTR) e migliorando la qualità percepita dall'utente.

Dal punto di vista teorico, il problema che ho affrontato rientra in pieno nel dominio della Text Classification, che è una delle principali aree del Natural Language Processing (NLP). Storicamente, lo smistamento dei ticket in azienda è sempre stato gestito da utenti o tramite sistemi software rule-based, cioè fondati su costrutti logici rigidi del tipo IF-THEN. Tuttavia, studiando la teoria e come evidenziato in letteratura (Gaddis, 2016), ho capito che questi approcci risultano fragili in produzione perché faticano a gestire l'ambiguità del nostro linguaggio naturale, non riconoscono i sinonimi e sono vulnerabili ai refusi di battitura.

Per superare questi limiti strutturali, ho optato per un approccio moderno basato sul Machine Learning supervisionato. Invece di programmare infinite istruzioni esplicite su cosa cercare nel testo, ho permesso all'algoritmo di apprendere autonomamente le correlazioni statistiche tra i termini utilizzati dagli utenti e le categorie di destinazione. Per farlo, ovviamente, l'ho dovuto addestrare su un dataset reale e preventivamente etichettato.

Entrando nel vivo delle metodologie e delle mie scelte implementative, ho dovuto trasformare i dati testuali in un formato che la macchina potesse effettivamente elaborare. I modelli di Machine Learning non leggono parole ma richiedono in input vettori numerici. Per la vettorizzazione del testo, ho scelto la tecnica TF-IDF, ovvero la Term Frequency - Inverse Document Frequency. Rispetto a un più semplice ma limitato approccio Bag of Words, che si basa sul conteggio di quante volte compare una parola, il TF-IDF fornisce una rappresentazione vettoriale pesata e intelligente, andando a valutare la reale rilevanza di un termine nel singolo documento rispetto all'intero corpus a disposizione (Zinoviev, 2017). La formula matematica che il mio codice utilizza per calcolare questo peso statistico è la seguente: $w_{i,j} = tf_{i,j} \times \log(N / df_i)$. Proprio il termine logaritmico $\log(N / df_i)$ svolge un ruolo fondamentale in questo processo perché penalizza matematicamente le parole molto frequenti ma totalmente non informative (come gli articoli o termini troppo generici come “problema”), assegnando invece un peso maggiore e deciso a parole specifiche e discriminanti, come possono essere “fattura”, “server” o “python”. Inoltre, per preservare il contesto locale della frase che l'utente ha scritto, ho introdotto nella pipeline l'analisi dei bigrammi, cioè le coppie di parole consecutive. È un trucco essenziale per far cogliere all'algoritmo la differenza semantica tra espressioni all'apparenza simili come “non funziona” e “funziona”.

Una volta vettorizzati i testi in uno spazio ad alta dimensionalità, dovevo scegliere un classificatore e per questo task ho optato per la Regressione Logistica. L'ho preferita rispetto ad algoritmi basati puramente su margini geometrici come le Support Vector Machines (SVM), perché la Regressione Logistica modella esplicitamente le probabilità di appartenenza a una determinata classe. In un contesto produttivo come quella del mio elaborato, questa caratteristica è preziosa poiché permette di fornire all'utente non solo una predizione secca, ma un vero e proprio grado di confidenza (ad esempio stampando a schermo un “Priorità Alta al 92%”). L'algoritmo mi garantisce inoltre una bassissima complessità temporale ed un'elevata velocità di inferenza, vitale per la reattività della web app.

Infine, per ottimizzare al massimo le risorse di calcolo e la memoria RAM, ho incapsulato l'intero modello in un **MultiOutputClassifier** (Maggi, 2020), il che mi ha permesso di calcolare simultaneamente sia la Categoria che la Priorità del ticket. Tutto questo lavoro, unito all'integrazione delle tecniche di Explainable AI tramite la libreria LIME, ha reso il mio modello totalmente ispezionabile. Ora l'interfaccia permette di evidenziare visivamente all'utente quali esatti termini testuali abbiano influenzato la classificazione, garantendo una necessaria trasparenza.

Descrizione dei principali aspetti progettuali
(Sviluppare l'elaborato richiesto dalla traccia prescelta):

Lo sviluppo pratico di AutoTriage NLP si basa su un'architettura modulare scritta interamente in Python, pensata per gestire l'intero ciclo di vita del dato, partendo dall'acquisizione grezza fino ad arrivare all'inferenza dell'algoritmo. Per mantenere il codice ordinato e soprattutto manutenibile, ho deciso di strutturare il progetto in tre macro-aree logiche ben distinte: il Data Engineering, il Modello di Machine Learning vero e proprio e infine il Frontend per l'utente.

La prima fase di **Data Engineering** e preparazione dei dati ha riguardato la costruzione di un dataset di addestramento in lingua italiana che fosse qualitativamente valido. Dovendo tradurre oltre 20.000 ticket tramite lo script **translate_data.py**, mi sono subito reso conto che un approccio sequenziale sarebbe stato troppo lento. Ho quindi scritto uno script che parallelizzasse le chiamate alle API di Google Translate sfruttando la potenza del **ThreadPoolExecutor**. Tuttavia, interrogando i server in modo massiccio, ho dovuto gestire eventuali interruzioni o blocchi temporanei delle API dovute al rate limiting. Ho risolto inserendo strategicamente dei ritardi programmati tra le richieste e implementando un salvataggio intermedio, essenziale per riprendere la traduzione esattamente dal punto di crash senza perdere i dati già elaborati. In parallelo, ho affidato allo script **prepare_data.py** la normalizzazione e pulizia dei dati. Questo file verifica in automatico la presenza del dataset tradotto e si preoccupa di mappare le vecchie etichette originali di Kaggle verso le tre macro-categorie che ho definito per il progetto, ovvero Amministrazione, Tecnico e Commerciale, utilizzando dei semplici dizionari di conversione. Per far capire meglio come ho riorganizzato i dati, ecco come ho strutturato nel codice la mappatura delle categorie di Kaggle verso queste tre macro-classi aziendali del progetto:

```
# Mappatura per ridurre le categorie originali di Kaggle alle mie 3 macro-classi
CATEGORY_MAP: Dict[str, str] = {
    "Technical Support": "Tecnico",
    "IT Support": "Tecnico",
    "Service Outages and Maintenance": "Tecnico",
    "Product Support": "Tecnico",
    "Billing and Payments": "Amministrativo",
    "Returns and Exchanges": "Amministrativo",
    "Human Resources": "Amministrativo",
    "General Inquiry": "Commerciale",
    "Customer Service": "Commerciale",
    "Sales and Pre-Sales": "Commerciale"
}
```

Analizzando il dataset estratto da Kaggle, ho riscontrato la presenza di rumore testuale, come ritorni a capo, doppi spazi e firme automatiche dei clienti di posta, che avrebbe compromesso e falsato l'efficacia della vettorizzazione TF-IDF. Per normalizzare queste stringhe prima della traduzione, ho dovuto implementare alcune espressioni regolari (Regex) utilizzando la libreria **re** nativa di Python. Successivamente, ho applicato un lowercasing forzato sull'intero testo. Questo passaggio ha impedito al modello di classificare varianti come "Server" con la maiuscola e "server" con la minuscola come se fossero token distinti, ottimizzando così la dimensione complessiva del vocabolario. Inoltre, durante le prime fasi di test, è emerso un limite legato alle liste di stop-words standard, classiche formule di cortesia molto frequenti nelle mail, come "salve", "grazie" o "cordiali saluti", ottenevano un peso statistico elevato pur non apportando alcun reale valore semantico ai fini della classificazione dei reparti. Per correggere questa distorsione, ho dovuto estendere la configurazione NLP di base creando un dizionario custom di stop-words in italiano, dal quale ho escluso esplicitamente tutti i convenevoli per forzare l'algoritmo matematico a valutare esclusivamente la terminologia tecnica.

Risolto il problema dei dati, mi sono dedicato all'architettura del **Modello di Machine Learning** all'interno del file **unified_model.py**. Per la componente predittiva, ho creato una classe **UnifiedModel** capace di racchiudere in un'unica pipeline sia la fase di vettorizzazione che quella di classificazione.

La trasformazione del testo grezzo in vettori numerici tramite la vettorizzazione TF-IDF tiene conto non solo delle singole parole slegate (gli unigrammi) ma anche dei bigrammi, ovvero le coppie di parole. È una strategia che ho ritenuto vitale per non perdere il senso del contesto locale della frase, permettendo al mio algoritmo di distinguere la differenza che c'è tra “funziona” e un “non funziona”. A livello di classificazione pura, piuttosto che gestire due modelli separati, ho usato la classe **MultiOutputClassifier** fornita da Scikit-Learn. Questo modulo mi ha consentito di addestrare due repressori logistici in modo completamente parallelo, uno focalizzato sulla Categoria e l'altro sulla Priorità, il tutto all'interno della stessa esecuzione, abbattendo i tempi di inferenza.

A livello di codice, ho tradotto questa architettura concettuale creando, di fatto, una singola **Pipeline** all'interno della classe **UnifiedModel**, capace di gestire sia la vettorizzazione che la complessa classificazione.

```
# Costruisco la pipeline
self.pipeline = Pipeline([

    # 1. Vettorizzazione
    # Uso max_features=5000 per non appesantire il modello e n_gram=(1,2)
    # per catturare concetti come "non funziona"
    ('tfidf', TfidfVectorizer(
        stop_words=self.stopwords,
        max_features=5000,
        ngram_range=(1, 2)
    )),

    # 2. Classificazione Multipla
    # Il class_weight='balanced' mi aiuta a gestire le classi minoritarie
    ('clf', MultiOutputClassifier(
        LogisticRegression(
            solver='lbfgs',
            max_iter=1000,
            random_state=42,
            class_weight='balanced'
        )
    ))

])
```

Per pianificare il tutto ho scritto lo script **train_unified_model.py**, che gestisce proprio la fase di addestramento vero e proprio e di serializzazione. Al suo interno, il codice suddivide il dataset appena pre-processato affidandosi a un classico split 80/20 per il Training Set e il Test Set, un passo obbligato per evitare che il modello impari i dati a memoria e vada in overfitting. Una volta addestrato il modello e calcolate le metriche di validazione, ho fatto in modo che lo script utilizzasse la libreria **joblib** per serializzare e salvare il modello all'interno di un file binario **.pkl**. Facendo così, ho reso il modello matematico totalmente persistente e può essere caricato direttamente dalla web app senza doverlo riaddestrare da zero a ogni singola esecuzione dell'utente.

Spostandomi poi sul **Frontend**, che ho sviluppato interamente con Streamlit nel file **app.py**, ho voluto chiarire che non l'ho mai concepito come una semplice vetrina estetica, ma vi ho integrato delle vere e proprie logiche di business avanzate.

Tra queste spicca sicuramente la priorità ibrida. Per evitare che il modello statistico sottostimasse ticket potenzialmente critici per un'azienda, ho affiancato all'output del Machine Learning un rigido controllo deterministico basato su delle specifiche keyword. In pratica, se la macchina scansiona il testo del ticket e rileva parole altamente sensibili e pericolose, come “hacker”, “fermo” o “scadenza”, il mio applicativo ignora le probabilità e forza matematicamente la priorità su “Alta”, garantendo così un livello di sicurezza aggiuntivo. Per dimostrare a livello di codice come funziona questo blocco di sicurezza, ecco un estratto della funzione **calculate_priority** dal file **app.py**, dove si vede esplicitamente la lista delle parole chiave “critiche” che ho definito per scavalcare l'algoritmo e forzare l'allarme per l'urgenza.

```
# Dizionari per il controllo regole
critical_keywords = [
    "virus", "hacker", "attacco", "violazione", "perso dati", "cancellato",
    "fermo", "blocco", "bloccato", "scadenza", "entro domani", "urgent",
    "subito", "velocemente", "critico", "panico", "terribile"
]
dampening_keywords = [
    "con calma", "non è urgente", "non urgente", "nessuna fretta",
    "quando potete", "appena possibile", "senza urgenza",
    "normale", "nessun problema", "funziona", "tutto ok", "informazione"
]

text_lower = text.lower()
is_critical = any(kw in text_lower for kw in critical_keywords)
is_dampener = any(kw in text_lower for kw in dampening_keywords)

# Controllo 1: Presenza di termini molto critici senza formule di cortesia/calma
if is_critical and not is_dampener:
    trigger_word = next((kw for kw in critical_keywords if kw in text_lower), "keyword")
    debug_probs['Alta'] = 0.99
    debug_probs['Media'] = 0.01
    debug_probs['Bassa'] = 0.00
    return 'Alta', 0.99, debug_probs, [trigger_word]
```

Per rendere le decisioni del modello pienamente verificabili, ho integrato un modulo direttamente nella dashboard basato sulla libreria LIME. Così facendo, ogni volta che viene analizzato un nuovo ticket, l'interfaccia di Streamlit genera un grafico esplicativo che mostra all'utente esattamente quali parole specifiche hanno influenzato maggiormente l'assegnazione finale della categoria, rendendo il comportamento dell'algoritmo assolutamente trasparente.

Infine, per convalidare la scelta di utilizzare un dataset reale (che è più costoso in termini di tempo e di preparazione) rispetto all'usare uno generato artificialmente, ho implementato un intero modulo di sperimentazione composto da due script ausiliari. Il primo, **generate_synthetic_data.py**, è un rigoroso script procedurale che genera una baseline di 500 ticket fittizi basandosi unicamente su dei template predefiniti. Il secondo, **compare_models.py**, addestra in modo del tutto asettico due versioni distinte del modello, una sui dati sintetici e l'altra sui dati reali. Finito l'addestramento, il codice esegue una valutazione incrociata, andando a testare il modello sintetico sui dati reali e viceversa.

I risultati di questo test, che ho poi salvato e renderizzato come matrici di confusione direttamente all'interno della cartella **assets/img/png**, dimostrano che il modello addestrato sui dati veri ha sviluppato una capacità di generalizzazione nettamente superiore. È proprio l'esito di questo test comparativo a giustificare tutto il tempo che ho dedicato durante le prime settimane alla fase iniziale di Data Engineering.

Campi di applicazione

(Descrivere gli ambiti di applicazione dell'elaborato progettuale e i vantaggi derivanti della sua applicazione):

Sebbene il progetto AutoTriage NLP sia stato sviluppato pensando alle esigenze di un Help Desk IT, fin dalle prime righe di codice ho concepito l'intera architettura software per essere totalmente indipendente dal dominio applicativo. L'idea era che, sfruttando il binomio tra la vettorizzazione TF-IDF e la Regressione Logistica, il sistema potesse essere riadattato a contesti operativi completamente diversi semplicemente fornendo un nuovo dataset di addestramento. Tutto questo senza la minima necessità di dover modificare la logica del codice sottostante o di dover riscrivere interi moduli.

L'implementazione reale di un sistema di triage basato sul Machine Learning permette a un'azienda di fare un vero salto di qualità, passando da una gestione reattiva a un approccio proattivo, portando a vantaggi organizzativi che si possono misurare concretamente. Innanzitutto, si ottiene una drastica riduzione dei tempi di risoluzione (TTR). L'assegnazione istantanea del ticket al reparto di competenza elimina completamente i colli di bottiglia legati allo smistamento manuale che un utente deve fare leggendo i testi riga per riga. A questo si aggiunge l'enorme vantaggio della continuità del servizio. Essendo un classificatore automatico, non è soggetto a limiti di orario, garantendo una copertura totale H24 e gestendo in maniera immediata gli improvvisi picchi di richieste. Un altro aspetto fondamentale è il rispetto degli SLA (Service Level Agreements). L'identificazione immediata delle criticità, supportata da quelle logiche ibride basate su keyword che ho implementato nel codice, previene violazioni delle tempistiche contrattuali. Infine, non va sottovalutata l'ottimizzazione delle risorse umane. Demandando alla macchina le attività di classificazione, il personale può essere riallocato su compiti a maggior valore aggiunto, ovvero quelli che richiedono empatia o capacità decisionali complesse che un algoritmo non potrà replicare.

Per dimostrare la versatilità di questo approccio supervisionato, ho analizzato cinque possibili scenari in cui la stessa identica architettura che ho programmato potrebbe essere impiegata con successo.

Pensiamo ad esempio al mondo degli e-commerce, per automatizzare il supporto clienti. Addestrando il classificatore su uno storico di chat ed email scaricate dal database aziendale, il sistema può imparare a distinguere le richieste informative di base, contenenti termini come “tracking” o “spedizione”, dalle vere e proprie criticità operative, come le email che menzionano “reso”, “rimborso” o “merce danneggiata”. Questo permette di far partire risposte automatiche per chi cerca solo il pacco e di instradare subito i clienti insoddisfatti verso gli operatori fisici.

Un altro ambiente perfetto è il settore Bancario e Assicurativo per la gestione documentale. Facendo analizzare all’algoritmo i testi delle PEC e delle comunicazioni formali, si possono facilmente separare le pratiche amministrative standard, come l’aggiornamento anagrafico, dai veri eventi critici che richiedono tempistiche legali rigorose. Riconoscere parole come “sinistro”, “frode” o “blocco carta” riduce enormemente il rischio di pesanti sanzioni per l’istituto.

Uscendo dall’ambito puramente commerciale, il modello potrebbe persino trovare posto nella Sanità e nella Telemedicina come triage digitale preventivo. Ovviamente l’obiettivo non è mai sostituire il giudizio clinico di un medico, ma il software può tranquillamente agire da primo filtro semantico sui portali di prenotazione. Riconoscendo e isolando i termini burocratici come “ricetta” o “certificato”, li separa nettamente dalle descrizioni sintomatologiche, caratterizzate da parole come “dolore” o “febbre”, potendo così assegnare automaticamente un codice di priorità per accelerare la presa in carico da parte del personale sanitario.

Tornando al business, nel Marketing B2B questo codice è utile per la qualificazione dei Lead. Facendogli analizzare il campo testuale dei form di contatto su siti web, l’algoritmo impara in fretta a distinguere i messaggi con una reale intenzione d’acquisto, individuando i termini come “preventivo”, “budget” o “incontro”, dalle richieste generiche o dal puro spam. Questo permette alla forza vendita di non perdere tempo e concentrarsi solo ed esclusivamente sui contatti più promettenti.

Infine, c’è il vasto mondo dell’analisi della reputazione, nota anche come Sentiment Analysis. Cambiando leggermente il focus della classificazione dal puro argomento testuale alla “polarità” emotiva del testo, il modello può analizzare le recensioni online. Assegnando dei pesi matematici elevati a termini critici come “truffa”, “pessimo” o “vergogna”, il sistema può generare alert in tempo reale direttamente sulla dashboard di Streamlit, permettendo all’azienda di intervenire tempestivamente per arginare e gestire eventuali crisi d’immagine prima che diventino virali.

In sintesi, credo che questo progetto dimostri come l’elaborazione del linguaggio naturale possa tradursi in uno strumento applicativo concreto. L’obiettivo finale che ho raggiunto al termine di queste settimane di sviluppo non si limita alla stesura di un algoritmo funzionante, ma consiste nella realizzazione di un’architettura software fortemente scalabile e interpretabile, capace di ottimizzare la gestione delle informazioni non strutturate e di supportare attivamente i processi decisionali all’interno di una vera organizzazione aziendale.

Valutazione dei risultati

(Descrivere le potenzialità e i limiti ai quali i risultati dell’elaborato sono potenzialmente esposti):

L’ultima vera fase del mio progetto l’ho dedicata alla validazione dell’ipotesi iniziale che mi ero posto. Volevo verificare sul campo se un algoritmo tradizionale e tutto sommato “leggero” come la Regressione Logistica potesse gestire efficacemente la complessa classificazione dei ticket, a patto ovviamente di operare su dati pre-processati correttamente. Per farlo, ho condotto tutti i test finali sul Test Set, che ho isolato corrispondendo al 20’% del corpus totale, ovvero circa 4.000 ticket completamente inediti per il modello. Valutare le performance su un dataset reale, seppur tradotto in automatico, fornisce una metrica di base, o baseline, che ritengo molto più vicina a un reale scenario produttivo aziendale rispetto ai risultati, spesso irrealisticamente alti, che si ottengono in ambito didattico lavorando su dati fittizi.

Proprio per dimostrare l’importanza di utilizzare dati reali fin dall’inizio, ho deciso di condurre un test comparativo, un vero e proprio A/B Test, tra due versioni distinte dell’algoritmo. Da una parte ho creato il Modello A, addestrato su un campione di dati sintetici che ho generato tramite dei template; dall’altra il Modello B, che è il mio progetto principale addestrato sul dataset Kaggle.

Tabella Riassuntiva delle Performance

Task di Classificazione	Accuracy (Globale)	Precision	Recall	F1-Score
Categoria del Ticket	59%	58%	59%	57%
Priorità del Ticket	46%	45%	46%	44%

Per avere un quadro più preciso, la tabella seguente mostra la reale capacità del mio modello di distinguere tra i vari reparti aziendali.

Dettaglio per Classe (F1-Score)

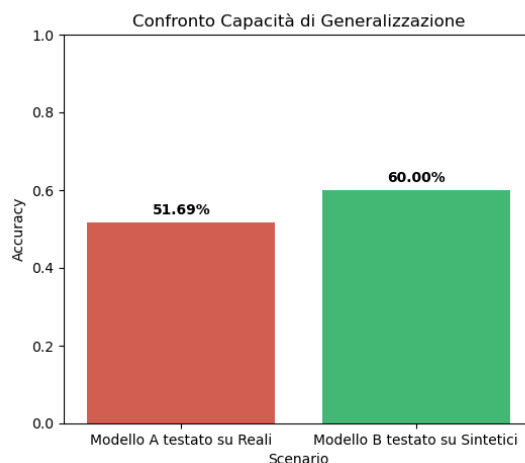
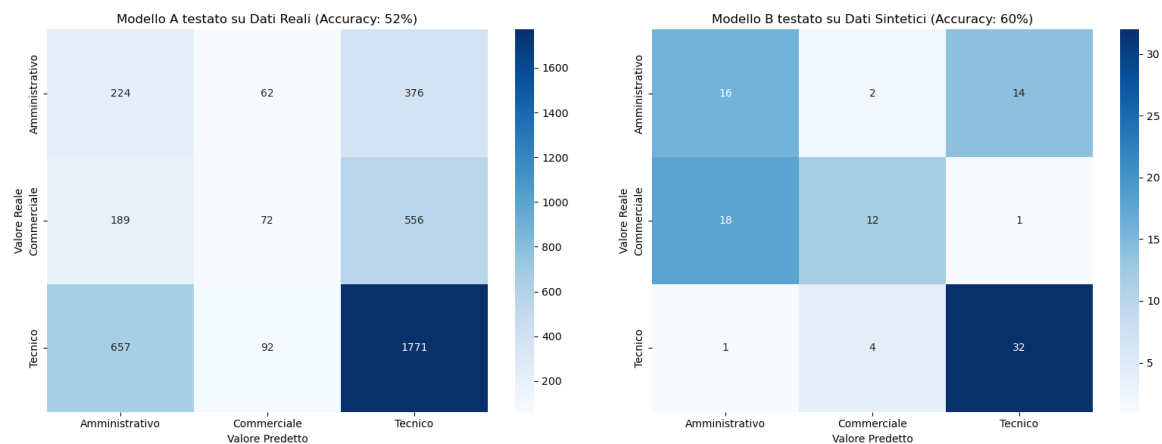
Categoria	F1-Score	Note
Tecnico	0.69	La classe più performante grazie a una terminologia specifica e discriminante.
Amministrativo	0.51	Risente di sovrapposizioni semantiche con l’ambito commerciale.
Commerciale	0.41	Classe con maggiore variabilità lessicale nel dataset di training.

Tutti i dati che presento in questa sezione derivano dall'ultima validazione effettuata a riga di comando sul Test Set. Per garantire che l'intero esperimento fosse scientificamente riproducibile e che i risultati non fossero casuali, ho fissato il seed impostando il parametro **random_state** a **42**, in modo che lo split dei dati e le epoche di training rimanessero costanti a ogni esecuzione. Guardando questi numeri e analizzando nel dettaglio i falsi positivi e i falsi negativi emersi dalle matrici di confusione generate dallo script **compare_models.py**, mi sono posto alcune domande sul perché l'algoritmo facesse molta più fatica su specifiche categorie. Mentre la classe "Tecnico" viene riconosciuta con grande facilità e precisione, probabilmente perché l'uso di termini univoci come "router", "database" o "schermata blu" non lasciano spazio a grosse interpretazioni, ho notato una forte e fastidiosa "zona grigia" tra i ticket classificati come "Amministrativo" e quelli etichettati come "Commerciale".

Andando a leggere manualmente, riga per riga, alcuni dei ticket testuali che il modello ha predetto in modo errato, ho capito che l'errore non era affatto dovuto a un bug del codice o a una svista nel preprocessing, ma derivava dalla natura intrinsecamente ambigua dei processi aziendali stessi. Molto spesso, infatti, accade che un agente di vendita (quindi reparto Commerciale) apra un ticket interno per chiedere spiegazioni su una fattura errata inviata a un suo cliente (questione di pertinenza dell'Amministrazione). In questi casi limite, il testo del ticket contiene un mix linguistico perfetto di vocaboli appartenenti a entrambe le sfere semantiche, presentando contemporaneamente parole come "preventivo", "cliente Rossi", "storno" e "bonifico". In queste situazioni critiche, l'algoritmo TF-IDF va inevitabilmente in difficoltà perché il peso matematico dei termini si bilancia quasi perfettamente. Questo fenomeno, noto nel Machine Learning come class overlapping, ovvero la sovrapposizione delle classi, mi ha dimostrato ancora una volta l'importanza di avere sempre un operatore umano a supervisione del sistema. Ho capito che il modello non deve mai essere visto come un decisore assoluto e infallibile, ma piuttosto come un preziosissimo "copilota" digitale che smaltisce in autonomia l'80% del lavoro banale e ripetitivo. Così facendo, lascia le richieste ambigue, cioè quella minoranza di ticket in cui le probabilità calcolate dal modello per due categorie sono molto vicine (ad esempio un 48% contro un 52%), all'analisi logica e all'intuito insostituibile dell'operatore umano.

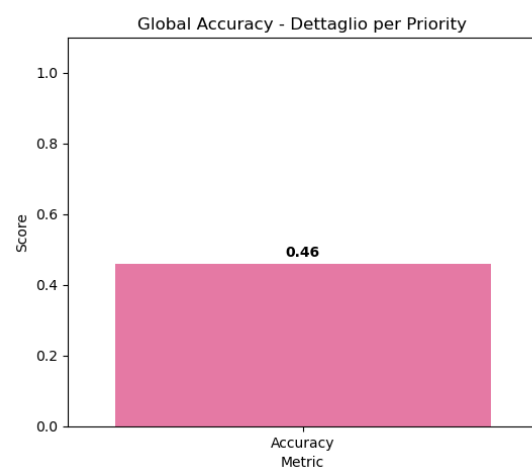
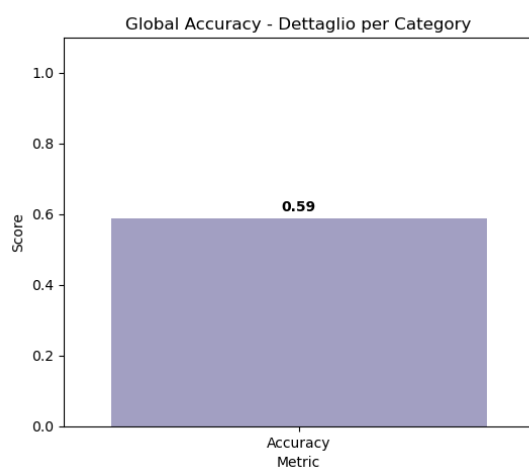
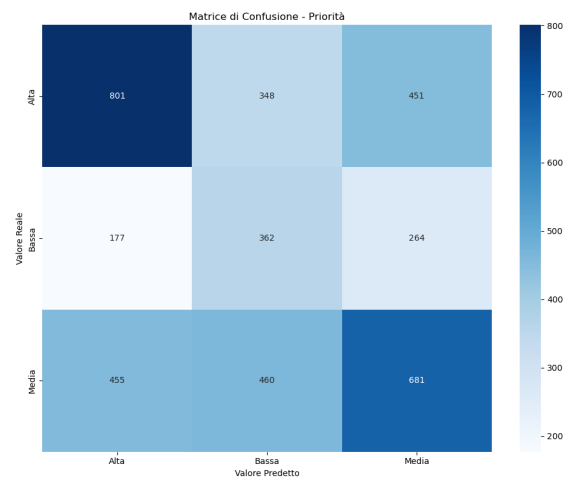
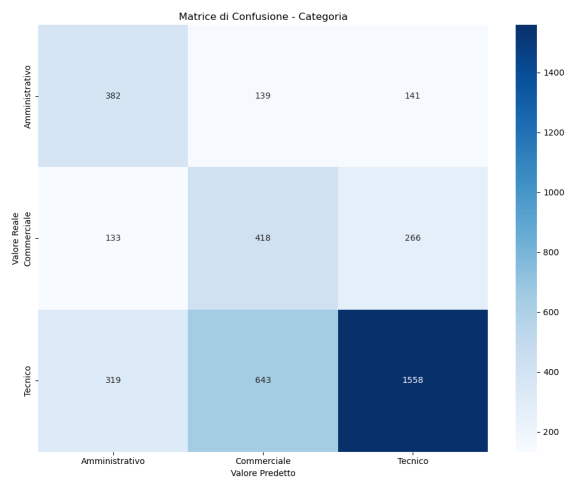
A questo punto, vorrei fare una doverosa precisazione tecnica sul dataset. Avendo tradotto i testi originali in batch tramite le API di Google Translate, far ripartire l'intera pipeline di traduzione da zero oggi potrebbe produrre metriche leggermente diverse da quelle che ho ottenuto. Questo accade perché il motore neurale di Google si aggiorna di continuo e, interrogandolo nuovamente, potrebbe restituire sinonimi o formulazioni differenti rispetto a quando ho scaricato io i dati, alterando di fatto la distribuzione del vocabolario su cui si basa il calcolo dei pesi. Per questo motivo, ci tengo a sottolineare che i grafici e i numeri che documento in questo elaborato fanno riferimento allo snapshot esatto del modello addestrato sul file locale **tickets_it_augmented.csv** che ho storicizzato in fase di sviluppo.

Guardando ai risultati finali sul Test Set, l'accuratezza globale si attesta al 59% per la scelta della Categoria e al 46% per la Priorità. Di seguito ho inserito le matrici di confusione per dare un'idea molto più chiara e visiva di come si distribuiscono realmente le predizioni e dove il codice tende a confondersi maggiormente.

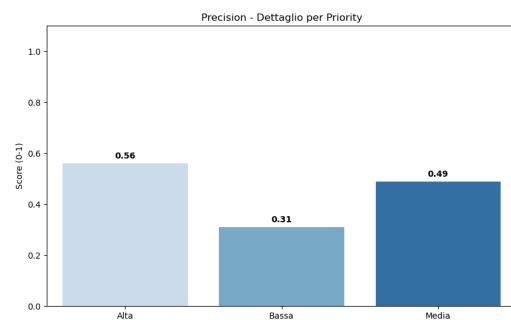
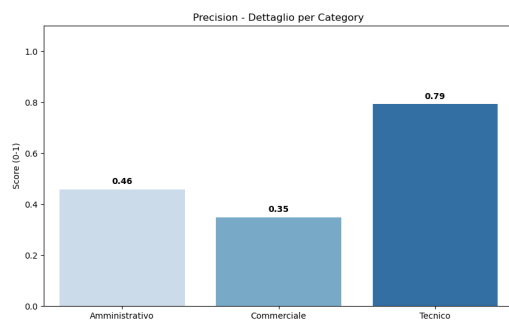


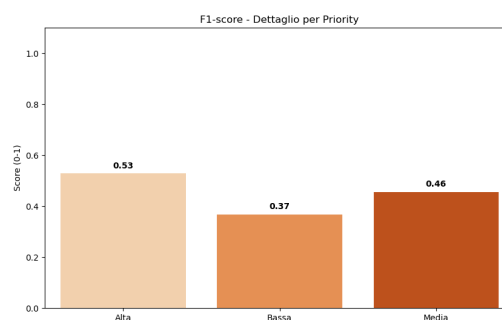
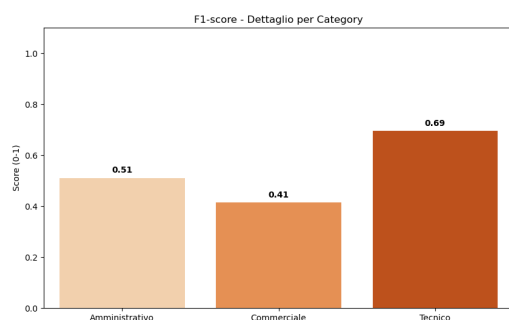
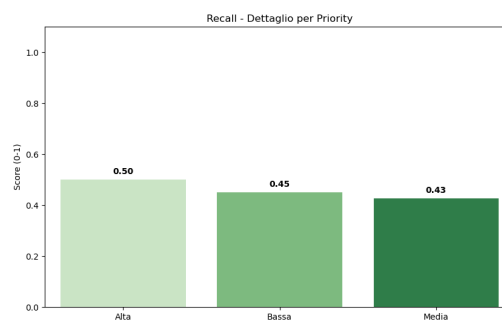
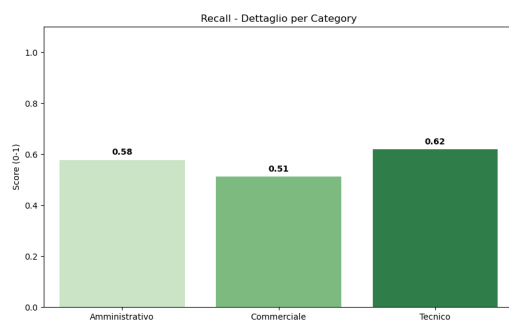
Analizzando i grafici, i risultati evidenziano un netto divario tra i due approcci. Il Modello A ha registrato un vero e proprio crollo delle performance, precipitando a un'accuratezza di circa il 52% quando è stato testato sui ticket reali. Questo calo così drastico è l'esempio da manuale dell'overfitting. Il modello sintetico ha imparato a riconoscere pattern fissi e keyword esatte inserite nei template, ma non è in grado di generalizzare quando esce dalla sua comfort zone. Al contrario, il Modello B, che è il cuore del mio lavoro, si è dimostrato immensamente più robusto, raggiungendo un'accuratezza del 60% anche quando è stato testato sui dati sintetici che non aveva mai visto in fase di training. Addestrare l'algoritmo sul "rumore" tipico del linguaggio umano, fatto di errori ortografici, sinonimi e ambiguità, lo ha costretto a faticare di più in fase di training, ma gli ha permesso di generalizzare in modo più efficace nel mondo reale.

L'analisi approfondita delle metriche diagnostiche restituisce un quadro assolutamente incoraggiante. Ho inserito questi primi grafici per avere un colpo d'occhio immediato sulle performance generali dell'applicativo, prima di passare ad analizzare le metriche scendendo nel dettaglio classe per classe.



Osservando nel dettaglio i valori di Precision e F1-Score per classe, emerge chiaramente che il modello ottiene i risultati nettamente migliori sulla categoria “Tecnico”, toccando un F1-Score di 0.69. la matrice di confusione mostra che la fastidiosa sovrapposizione tra classi semanticamente vicine, come appunto “Amministrativo” e “Commerciale”, viene comunque mitigata in modo molto soddisfacente dalla vettorizzazione TF-IDF, che penalizza i termini troppo comuni che genererebbero solo rumore.





Un vero punto di forza pratico dell'applicativo che ho sviluppato è l'utilizzo di una logica ibrida, per la complessa gestione delle priorità. Sapevo che fidarsi solo della statistica era rischioso, quindi alla classificazione probabilistica pura del modello ho affiancato una regola deterministica basata su specifiche parole chiave. In sostanza, se nel testo del ticket compaiono termini critici come “virus”, “hacker” o “fermo”, il sistema scavalca le probabilità e forza istantaneamente l'assegnazione a una Priorità Alta. Questo strato di controllo aggiuntivo l'ho scritto appositamente per evitare che eventuali falsi negativi generati dal modello facessero ignorare ticket urgentissimi all'azienda. A questo concetto si lega strettamente il tema vitale dell'interpretabilità. L'uso intensivo che ho fatto della libreria LIME permette all'utente finale di visualizzare a schermo, con assoluta esattezza, quali termini testuali abbiano influenzato la predizione della macchina. Fornire una spiegazione visiva e comprensibile del comportamento dell'algoritmo è un passaggio che reputo fondamentale per favorirne l'adozione reale e per instaurare un rapporto di fiducia tra gli utenti finali e il sistema.

Spostandoci a un livello puramente implementativo, unire il backend matematico di Machine Learning con il frontend interattivo di Streamlit non è stato per niente banale e ha richiesto un bel po' di ingegneria del software. Streamlit, per come è disegnato nativamente, ricarica in modo automatico l'intero script Python ogni volta che l'utente interagisce con un qualsiasi widget, ad esempio quando preme il bottone "Analizza". Inizialmente, questo comportamento faceva sì che il file del modello **.pkl**, che pesa svariati megabyte, venisse ricaricato nella memoria RAM da zero a ogni singolo click, rendendo l'interfaccia estremamente lenta e scattosa. Per risolvere questo grave collo di bottiglia prestazionale, mi sono dovuto documentare sui meccanismi di caching avanzati messi a disposizione dal framework. Ho implementato il decoratore **@st.cache_resource** posizionandolo esattamente sopra la funzione di caricamento del modello, in questo modo, l'oggetto **UnifiedModel** viene istanziato nella memoria RAM del server una sola volta, esclusivamente all'avvio dell'applicazione. Tutte le successive inferenze lanciate dall'utente non fanno altro che pescare direttamente il modello già in memoria, abbattendo i tempi di risposta dell'interfaccia da diversi secondi a pochi millisecondi. È stato un passaggio tecnico fondamentale per trasformare uno script accademico in un prototipo reattivo e realmente utilizzabile da un ipotetico operatore di Help Desk senza frustrazioni.

Nonostante gli ottimi risultati raggiunti e la soddisfazione personale, sono perfettamente consapevole che l'architettura che ho implementato presenti alcune limitazioni intrinseche, dovute alle scelte progettuali che ho dovuto prendere per rientrare nei tempi. Il primo ostacolo riguarda i **limiti dell'approccio Bag-of-Words e della tecnica TF-IDF**. Questa tecnica, per quanto performante, tratta le singole parole in modo del tutto indipendente, ignorando di fatto la sintassi generale della frase e le sequenzialità temporale del discorso. Di conseguenza, il modello fatica a interpretare frasi grammaticalmente complesse, doppie negazioni o forme di sarcasmo. L'adozione di architetture a reti neurali più moderne, magari basate su Transformer come BERT, risolverebbe questo problema testuale, ma un approccio del genere richiederebbe risorse computazionali e tempi di inferenza nettamente superiori. Un'altra debolezza insidiosa risiede nella **qualità della traduzione automatica** in sé. Poiché l'intero Training Set è stato tradotto interamente affidandomi alle API, è altamente probabile che alcuni fastidiosi artefatti linguistici o traduzioni troppo letterali abbiano introdotto un certo grado di "rumore" statistico nel modello (basti pensare al termine informatico "term", che spesso viene tradotto come la parola "termine" anziché come "condizione"). Infine, bisogna tenere in considerazione il **degrado fisiologico delle performance nel tempo**. Il modello che ho addestrato attualmente è congelato e statico. Se il vocabolario dell'azienda dovesse evolversi o cambiare, ad esempio con il lancio di nuovi prodotti software non presenti nel mio vocabolario di addestramento originale, le performance del classificatore tenderebbero a calare progressivamente. Questo rende assolutamente necessario, in ottica aziendale, pianificare dei cicli di ri-addestramento periodici per mantenere il sistema affidabile.

Per evolvere ulteriormente questo elaborato verso una soluzione applicabile in azienda, i prossimi step implementativi che ho in mente potrebbero includere l'introduzione della lemmatizzazione. Appoggiandomi a librerie di NLP avanzate come spaCy, potrei normalizzare tutti i termini riportandoli alla loro radice linguistica, riducendo le variazioni inutili e alleggerendo i dizionari. Inoltre, svilupperei sicuramente un solido meccanismo di feedback loop integrato nell'interfaccia web, in cui l'utente può correggere con un click le predizioni errate, permettendo così al sistema di apprendere continuamente dai propri sbagli in modo dinamico.

Infine, per contestualizzare al 100% il progetto dal punto di vista aziendale e dimostrarne la fattibilità economica, ho effettuato una stima di base del Ritorno sull'Investimento (ROI) operativo. Considerando un tempo medio di 2 minuti spesi da un operatore per leggere, interpretare e smistare un ticket manualmente, calcolando un volume ipotetico di 10.000 ticket mensili in ingresso e un costo orario aziendale di 25 € l'ora, l'automazione spinta di questo triage genererebbe un risparmio notevole. La formula matematica è semplice:

$$\text{Risparmio} = (2 / 60 \text{ ore}) \times 10.000 \text{ ticket} \times 25\text{€/h} \approx 8.333\text{€} / \text{mese}$$

Questo dato prettamente economico, unito all'efficacia tecnica dimostrata a schermo dai vari test di generalizzazione sul codice, conferma a pieno la mia ipotesi iniziale: l'implementazione di tecniche di Natural Language Processing per automatizzare il triage non è solo un esercizio tecnico, ma rappresenta una gigantesca e concreta opportunità per ottimizzare i costi operativi dei servizi di Help Desk.